

NAME: TANKISO MASOEBE

OPEERATING SYSTEM WK2(ADVA)

1.

Scenario Analysis: Multilevel Queue Scheduling

A computer system is running several types of processes, including system tasks, interactive applications, and background jobs. To manage these processes efficiently, the operating system uses a **multilevel queue scheduling** algorithm. Processes are divided into separate queues based on their purpose and priority. System processes are placed in the highest-priority queue, interactive applications are placed in the next queue, and background tasks are placed in the lowest-priority queue. Each queue can use a different scheduling method. For example, the system queue may use priority scheduling, while the interactive queue uses Round Robin to provide quick responses. The background queue may use First-Come, First-Served scheduling. The operating system gives higher-priority queues more CPU time, ensuring important tasks are completed quickly. This configuration improves system performance, responsiveness, and resource management. However, low-priority processes may experience starvation if higher-priority queues are continuously busy.

2. Concept Research: Priority Scheduling

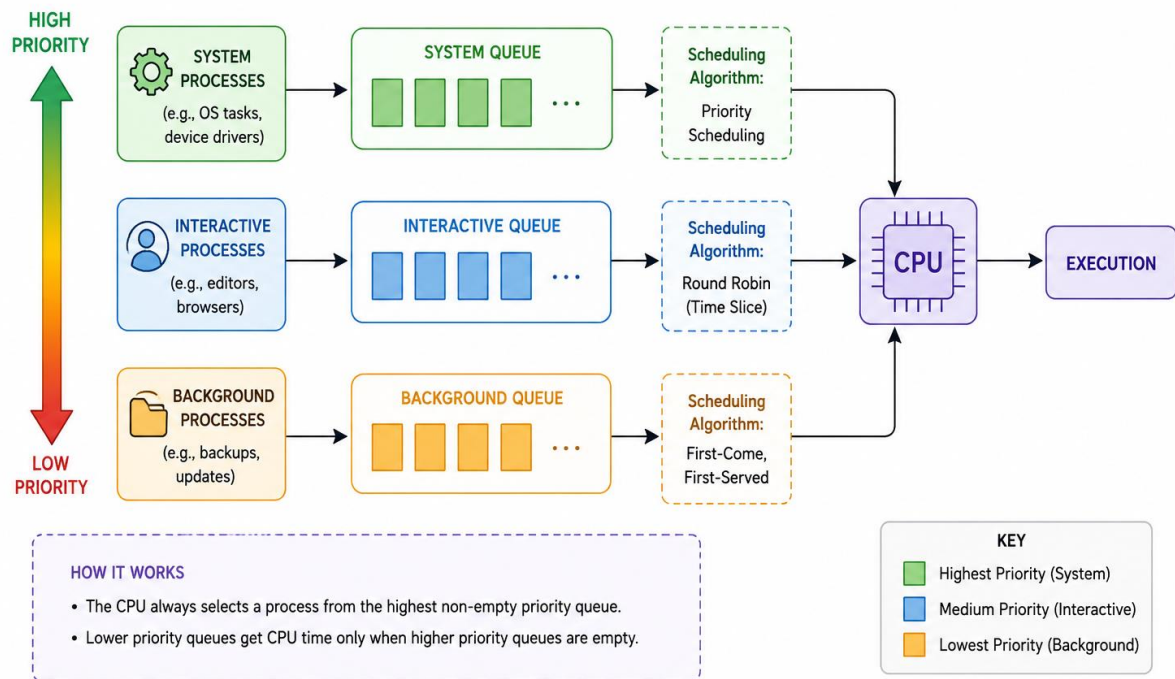
Priority scheduling is a CPU scheduling technique where processes are assigned different priority levels. The operating system selects the process with the highest priority to run first. It can be either preemptive, where a higher-priority process can interrupt a running process, or non-preemptive, where the current process continues until completion. Priority scheduling is useful for ensuring that important or urgent tasks receive CPU resources quickly. However, one major challenge is **starvation**, where low-priority processes may wait for a long time if high-priority processes continuously arrive. This can be reduced using **aging**, which gradually increases the priority of waiting processes. Priority scheduling also requires accurate priority assignment.

3. Tool Practice: Linux Scheduling Using `nice`

For this practical activity, I explored CPU scheduling in Linux using the `nice` command. The `nice` value controls the scheduling priority of a process. A process with a lower `nice` value receives higher priority, while a process with a higher `nice` value receives lower priority. I observed how changing the `nice` value can affect how CPU resources are shared between running processes. This activity helped me understand that scheduling is important for maintaining system performance and responsiveness. I also learned that administrators can adjust process priorities depending on the importance of tasks. Overall, the practical demonstrated how Linux manages CPU time efficiently among different processes.

MULTILEVEL QUEUE SCHEDULING

Each queue has its own scheduling algorithm and priority.

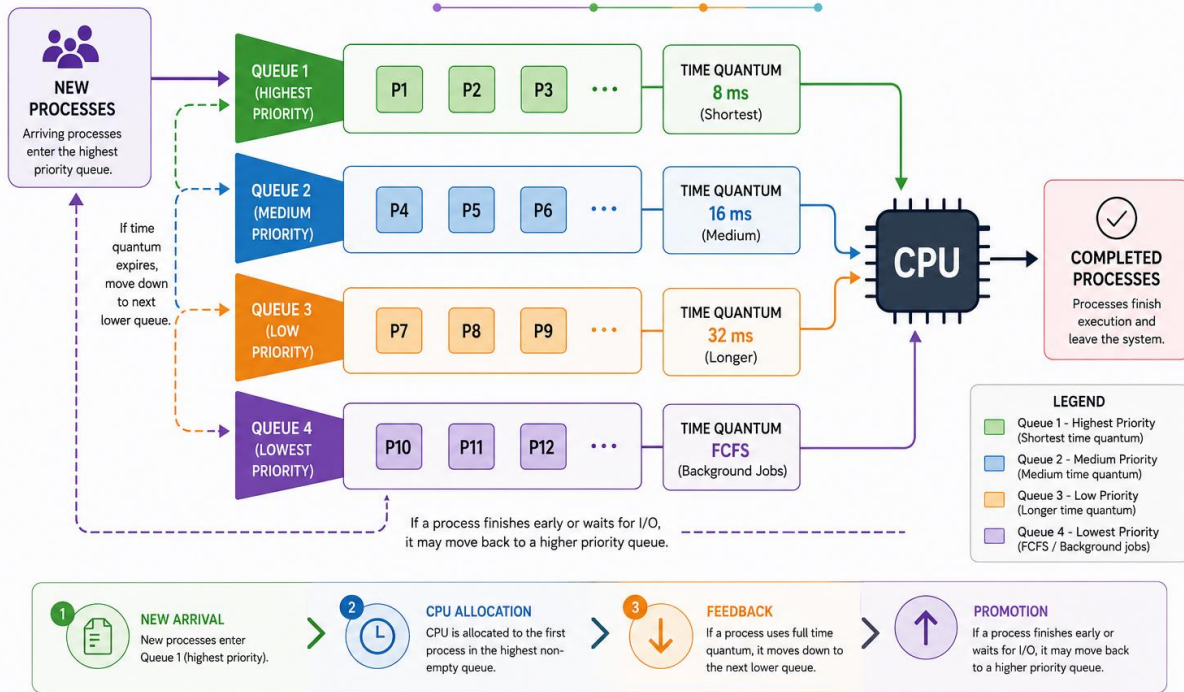


4. Reflection

Creating the multilevel feedback queue diagram helped me understand how process scheduling works in an operating system. I learned that processes are placed in different queues according to their behaviour and CPU requirements. Short and interactive processes receive higher priority and faster CPU access, while longer processes can move to lower queues with larger time limits. The feedback mechanism allows processes to move between queues, improving fairness and responsiveness. This activity also showed me how scheduling algorithms balance CPU utilisation and user experience. Overall, drawing the diagram made the concept easier to understand and helped me visualise how processes are managed.

MULTILEVEL FEEDBACK QUEUE SCHEDULING

Processes are scheduled dynamically based on priority and CPU usage.



Explanation

The Multilevel Feedback Queue scheduling algorithm uses several queues with different priority levels and time quantum. New processes usually enter the highest-priority queue, where they receive quick CPU access. If a process uses its entire time quantum without finishing, it is moved to a lower-priority queue with a longer time quantum. Processes that finish quickly or wait for input may remain in, or move back to, higher-priority queues. This approach improves response time for short and interactive tasks while still allowing longer background tasks to run. The feedback mechanism balances fairness, CPU efficiency, and system responsiveness in modern operating systems.